Análisis del Uso de la IA — ReMarket

IA como Atacante — Análisis de Escenarios de Amenaza

En este entregable se analizan tres escenarios donde un adversario utiliza modelos de Inteligencia Artificial generativa y herramientas automatizadas para descubrir o explotar debilidades en la arquitectura de **ReMarket**.

Escenario 1 — Spear-Phishing Hipercontextualizado Dirigido al Personal de Soporte

Pregunta	Respuesta Técnica Fundamentada
1. ¿Qué activo intenta comprometer?	Credenciales de acceso a la consola de administración y tokens de sesión de operadores de soporte técnico.
2. ¿Qué debilidad aprovecha?	El factor humano y la susceptibilidad al engaño mediante ingeniería social basada en urgencia operacional.
3. ¿Qué impacto podría producir?	Acceso ilegítimo al panel de control de usuarios, visualización de datos personales y posible escalada hacia infraestructura cloud.
4. ¿Qué controles de nuestra arquitectura reducen el riesgo?	**Autenticación multifactorial (MFA) resistente al phishing mediante llaves de hardware FIDO2 / WebAuthn (YubiKey)** en la VPN administrativa y consola; principio de mínimo privilegio en IAM (roles Just-in-Time sin permisos permanentes); y túnel VPN cifrado con mTLS.
5. ¿Qué evidencia permitiría detectarlo?	Intentos de inicio de sesión desde direcciones IP anómalas o fuera de geolocalización habitual en CloudWatch/GuardDuty; reportes de usuarios internos ante correos sospechosos; y discrepancias en el User-Agent.

Qué aporta la IA aquí que no aportaba el ataque tradicional:
La IA permite generar mensajes perfectamente personalizados, adaptando el lenguaje al manual de estilo interno de ReMarket y eliminando errores ortográficos o modismos que alertaban a los filtros tradicionales. Además, permite orquestar campañas masivas dirigidas a cientos de empleados en minutos a costo marginal cero.

Escenario 2 — Fuzzing Semántico y Descubrimiento de APIs con Fallas BOLA

Pregunta	Respuesta Técnica Fundamentada
1. ¿Qué activo intenta comprometer?	La información confidencial de pedidos (`POST /pedido`) y perfiles de clientes almacenados en la base de datos relacional.
2. ¿Qué debilidad aprovecha?	Fallas de autorización a nivel de objeto (*Broken Object Level Authorization* - OWASP API1:2023) o parámetros no documentados en endpoints REST.
3. ¿Qué impacto podría producir?	Exfiltración masiva de compras, direcciones y nombres de otros clientes al alterar secuencialmente los identificadores en las peticiones.
4. ¿Qué controles de nuestra arquitectura reducen el riesgo?	Validación estricta de esquemas OpenAPI en el **API Gateway**; autorización contextual en el microservicio de pedidos verificando que el `user_id` del token JWT firmado con ES256 coincida con el propietario del recurso; y rate-limiting en AWS WAF.
5. ¿Qué evidencia permitiría detectarlo?	Picos anómalos de respuestas HTTP 403 (Forbidden) y 404 en los registros de acceso de Envoy; ráfagas de solicitudes con IDs dispares desde una misma sesión registradas en OpenSearch SIEM.

Qué aporta la IA aquí:
En lugar de enviar cargas aleatorias fácilmente bloqueadas por firmas estáticas, los modelos de IA infieren la semántica y relaciones de la API leyendo la documentación pública o analizando respuestas JSON, generando secuencias de peticiones sintácticamente válidas pero lógicamente maliciosas.

Escenario 3 — Reconocimiento Automatizado de Secretos y Llaves en Código Fuente

Pregunta	Respuesta Técnica Fundamentada
1. ¿Qué activo intenta comprometer?	Claves maestras de API de pasarelas de pago (Stripe), certificados privados o credenciales de conexión a bases de datos.
2. ¿Qué debilidad aprovecha?	Fugas accidentales de secretos en commits de Git, fragmentos de código de prueba o dependencias públicas desactualizadas.
3. ¿Qué impacto podría producir?	Acceso administrativo directo a recursos en AWS o cobros fraudulentos sin necesidad de vulnerar la red perimetral.
4. ¿Qué controles de nuestra arquitectura reducen el riesgo?	Pre-commit hooks con **Gitleaks**; escaneo continuo con **TruffleHog**; custodia exclusiva en **AWS Secrets Manager / KMS**; y autenticación en CI/CD mediante **roles efímeros OIDC** (cero claves estáticas en el repositorio).
5. ¿Qué evidencia permitiría detectarlo?	Alertas de GitHub Secret Scanning y eventos inusuales en AWS CloudTrail invocando APIs desde IPs desconocidas con credenciales recién generadas.

Qué aporta la IA aquí:
Los analizadores convencionales dependen de expresiones regulares fijas (RegEx). Un agente de IA ofensivo utiliza comprensión del lenguaje para detectar secretos ofuscados, variables renombradas o claves incrustadas en cadenas codificadas en Base64/Hexadecimal que eluden los patrones sintácticos clásicos.

IA como Evaluador de Seguridad

Se presentó un resumen de la arquitectura de **ReMarket** a una IA para que encontrara riesgos, errores de diseño o controles faltantes, y evaluamos críticamente sus tres observaciones. El prompt describe la **versión inicial** del diseño (por eso aún menciona SPIRE y "SQS/Kafka"); los cambios que salieron de esta revisión ya están aplicados en el documento técnico.

Paso 1 — El Prompt Utilizado

```
Actúa como un Auditor Senior de Ciberseguridad Cloud y Criptografía Aplicada.
Evalúa el siguiente diseño de arquitectura de comercio electrónico (ReMarket):
- Microservicios en Amazon EKS con Istio Service Mesh, mTLS interno y certificados SPIFFE/SPIRE efímeros (24h).
- Perímetro protegido por Amazon CloudFront (CDN), AWS WAF y AWS Shield con TLS 1.3 (ECDHE-X25519) y HSTS Preload.
- API REST con JWT firmados en ES256 y llaves de idempotencia en ElastiCache Redis para POST /pedido.
- Datos en reposo protegidos con AES-256-GCM y Envelope Encryption mediante AWS KMS y CloudHSM (FIPS 140-2 L3).
- Contraseñas almacenadas exclusivamente con Argon2id (m=64MB, t=3, p=4) y salt CSPRNG único de 16 bytes.
- Pasarela de pagos integrada vía tokenización en el cliente con Stripe Elements (PCI-DSS SAQ-A), sin almacenamiento de
PAN/CVV en ReMarket. Webhooks de pago validados con firma HMAC-SHA256 y ventana anti-replay de 300 segundos.
- Facturas digitales firmadas con hash SHA-256 y ECDSA P-256 con certificado público X.509.
- Bus de eventos asíncrono en Amazon SQS/Kafka con cifrado SSE-KMS y políticas de mínimo privilegio IAM.
- Pipeline DevSecOps con pre-commit Gitleaks, SAST SonarQube, SCA Trivy/Snyk, firma de imágenes OCI con Sigstore Cosign y
despliegue GitOps con ArgoCD.

Identifica al menos tres observaciones técnicas: posibles vulnerabilidades no cubiertas, riesgos de implementación o
mejoras de endurecimiento criptográfico.
```

- **Herramienta:** Claude (Anthropic), usado a través de Claude Code
- **Fecha de consulta:** 6 de noviembre de 2026
- **Respuesta completa:** Anexo (al final de este documento)

Paso 2 — Evaluación Crítica de las Observaciones

Observación 1 — Autenticidad de los eventos en el bus

> *"Si un servicio con permiso de publicación es comprometido, puede inyectar un `PedidoPagado` falso y provocar un despacho sin cobro. Recomiendo: (a) separar temas o colas por tipo de evento y dar permiso de publicación sólo al servicio dueño de cada uno; (b) firmar los eventos críticos con una clave asimétrica de KMS y verificar la firma en el consumidor; (c) consumidores idempotentes y una cola de mensajes fallidos (DLQ)."*

Pregunta	Evaluación Crítica del Equipo
1. ¿Es técnicamente correcta?	**Sí.** SSE-KMS protege el dato guardado, no su origen, y una política que permite `Publish` no restringe el tipo de evento.
2. ¿Aplica a nuestra arquitectura?	**Sí.** Pedidos y Pagos publican en el mismo bus, e Inventario actúa sobre `PedidoPagado`.
3. ¿Qué riesgo intenta reducir?	Inyección de eventos falsos y despachos sin cobro (R6).
4. ¿La aceptamos, modificamos o rechazamos?	**Modificada.**
5. ¿Qué evidencia técnica justifica la decisión?	Adoptamos (b) y (c): `PedidoPagado` lo publica y firma sólo Pagos con una clave ECDSA P-256 de KMS que ningún otro rol puede usar, y los consumidores la verifican; consumidores idempotentes y DLQ. En lugar de (a), mantenemos un solo tema SNS: la firma ya impide que Pedidos falsifique un pago, y separar temas añade operación sin reducir más el riesgo. Descartamos Kafka y nos quedamos con SNS → SQS, así que la parte de mTLS en MSK no aplica.

Dónde queda reflejada: Sección 11, diagrama de arquitectura y riesgo R6.

Observación 2 — La ventana de 300 s no basta contra la repetición de webhooks

> *"Dentro de esos 5 minutos el mismo evento firmado puede llegar varias veces (Stripe incluso reintenta de forma legítima). Registren el `event.id` ya procesado e ignoren los repetidos. Además, antes de despachar, consulten el `PaymentIntent` en la API de Stripe y comparen monto y moneda con el pedido."*

Pregunta	Evaluación Crítica del Equipo
1. ¿Es técnicamente correcta?	**Sí.** La marca de tiempo limita *cuándo* se acepta un mensaje, no *cuántas veces*. La documentación de Stripe indica que un mismo evento puede entregarse más de una vez.
2. ¿Aplica a nuestra arquitectura?	**Sí.** Nuestro diseño sólo tenía firma + ventana.
3. ¿Qué riesgo intenta reducir?	Procesar dos veces un pago o confirmar un pedido con el pago de otro (R6).
4. ¿La aceptamos, modificamos o rechazamos?	**Aceptada.**
5. ¿Qué evidencia técnica justifica la decisión?	Guardar el `event.id` cuesta una escritura en Redis o en la BD por webhook. Comparar monto y moneda cierra el caso en que un pago real, pero de otro pedido o por menos dinero, se usa para confirmar una orden.

Dónde queda reflejada: Sección 9 (cuatro verificaciones del webhook) y matriz de riesgos (R6).

Observación 3 — Dónde guarda el navegador los JWT

> *"Si se guardan en `localStorage`, cualquier XSS puede leerlos y reutilizarlos desde otra máquina durante toda su vigencia. Recomiendo guardar la sesión en una cookie `HttpOnly; Secure; SameSite=Strict`, o bien mantener el token de acceso sólo en memoria con vida corta y el de refresco en una cookie `HttpOnly` con rotación en cada uso."*

Pregunta	Evaluación Crítica del Equipo
1. ¿Es técnicamente correcta?	**Sí.** JavaScript inyectado puede leer `localStorage`, pero no una cookie `HttpOnly`.
2. ¿Aplica a nuestra arquitectura?	**Sí, y además mostró una inconsistencia nuestra:** la clasificación de información decía "cookie" y la API decía `Authorization: Bearer`.
3. ¿Qué riesgo intenta reducir?	Robo de tokens de sesión mediante XSS (secuestro de sesión).
4. ¿La aceptamos, modificamos o rechazamos?	**Modificada.**
5. ¿Qué evidencia técnica justifica la decisión?	Elegimos la segunda opción: token de acceso de 15 min sólo en memoria, enviado como `Bearer`, y token de refresco en cookie `HttpOnly; Secure; SameSite=Strict` que rota en cada uso. Así la API sigue sin depender de cookies para cada petición (menos exposición a CSRF) y un XSS no puede llevarse una sesión duradera. La CSP estricta reduce además la probabilidad del XSS.

Dónde queda reflejada: Sección 2 (tokens de sesión) y sección 5 (`POST /pedido`).

Paso 3 — Decisión Final del Equipo

Observación Evaluada	Decisión Final	Cambio en el Proyecto
1. Autenticidad de eventos en el bus	**Modificada**	Firma ECDSA de `PedidoPagado` sólo por Pagos, idempotencia y DLQ; un solo tema SNS; Kafka descartado (sección 11).
2. Repetición de webhooks	**Aceptada**	Registro de `event.id` y comparación de monto y moneda con Stripe (sección 9).
3. Almacenamiento de JWT	**Modificada**	Acceso en memoria + refresco en cookie `HttpOnly` con rotación (secciones 2 y 5).

Lo que la respuesta no cubrió: no señaló que el prompt mezclaba dos opciones distintas (la CA de Istio y SPIRE), y como el prompt no incluía el diagrama, tampoco pudo ver que el webhook entraba directo a un pod privado. Esos errores aparecieron al comparar el diagrama con el texto: una IA sólo evalúa lo que se le muestra.

Reflexión Final: Límites de la Inteligencia Artificial en Ciberseguridad

Los LLMs ayudan a revisar diseños y proponer defensas, pero **una organización no debe aplicar automáticamente todo lo que recomienda una IA**:

1. **Contexto:** La IA sólo conoce lo que se le muestra; una recomendación correcta en general puede no aplicar al sistema concreto.
2. **Falsos positivos:** Los modelos marcan como vulnerables cosas que no lo son (una dependencia con una CVE en una función que nunca se llama, un “secreto” que es un ID público). Atenderlos sin verificar introduce cambios innecesarios y acostumbra al equipo a ignorar alertas.
3. **Alucinaciones:** Pueden inventar parámetros o configuraciones inexistentes con total convicción.
4. **Configuración real del sistema:** Una sugerencia pensada para un servidor tradicional puede ser imposible en un servicio gestionado (por ejemplo, mTLS en SQS) o romper la integración con Istio.
5. **Impacto operativo:** Reglas de WAF demasiado estrictas o firmas en cada lectura del catálogo pueden bloquear compras legítimas o tirar la tienda en temporada alta.
6. **Gestión de riesgos:** No existe la seguridad al 100%. Decidir qué riesgo se mitiga, transfiere o acepta depende del costo y del negocio, como hicimos en la sección 14; un modelo no tiene esa información.
7. **Responsabilidad humana:** PCI-DSS, la LFPDPPP o el GDPR responsabilizan a la empresa y a sus personas. Ningún auditor acepta “lo recomendó una IA” como justificación.

La IA es un **copiloto**: propone y acelera la revisión. La decisión final, su contexto y su responsabilidad son del equipo.

Anexo — Evidencia de consulta a IA — 01: Revisión de la arquitectura

Herramienta: Claude (Anthropic), usado a través de Claude Code (claude.ai/code) **Fecha:** 2026-11-06 **Quién la hizo:** Oscar David Hernández Rodríguez **Para qué sección:** Entregable 5 (IA como evaluador) y sección 16 del documento técnico

---

Prompt utilizado

```
Actúa como un Auditor Senior de Ciberseguridad Cloud y Criptografía Aplicada.
Evalúa el siguiente diseño de arquitectura de comercio electrónico (ReMarket):
- Microservicios en Amazon EKS con Istio Service Mesh, mTLS interno y certificados SPIFFE/SPIRE efímeros (24h).
- Perímetro protegido por Amazon CloudFront (CDN), AWS WAF y AWS Shield con TLS 1.3 (ECDHE-X25519) y HSTS Preload.
- API REST con JWT firmados en ES256 y llaves de idempotencia en ElastiCache Redis para POST /pedido.
- Datos en reposo protegidos con AES-256-GCM y Envelope Encryption mediante AWS KMS y CloudHSM (FIPS 140-2 L3).
- Contraseñas almacenadas exclusivamente con Argon2id (m=64MB, t=3, p=4) y salt CSPRNG único de 16 bytes.
- Pasarela de pagos integrada vía tokenización en el cliente con Stripe Elements (PCI-DSS SAQ-A), sin almacenamiento de
PAN/CVV en ReMarket. Webhooks de pago validados con firma HMAC-SHA256 y ventana anti-replay de 300 segundos.
- Facturas digitales firmadas con hash SHA-256 y ECDSA P-256 con certificado público X.509.
- Bus de eventos asíncrono en Amazon SQS/Kafka con cifrado SSE-KMS y políticas de mínimo privilegio IAM.
- Pipeline DevSecOps con pre-commit Gitleaks, SAST SonarQube, SCA Trivy/Snyk, firma de imágenes OCI con Sigstore Cosign y
despliegue GitOps con ArgoCD.

Identifica al menos tres observaciones técnicas: posibles vulnerabilidades no cubiertas, riesgos de implementación o
mejoras de endurecimiento criptográfico.
```

Respuesta de la IA (completa)

> **Observación 1 — Autenticidad de los eventos en el bus.** SSE-KMS sólo protege los mensajes mientras están guardados, y una política IAM de "mínimo privilegio" que permite publicar en la cola no distingue *qué* evento publica cada servicio. Si un servicio con permiso de publicación es comprometido, puede inyectar un `PedidoPagado` falso y provocar un despacho sin cobro. Recomiendo: (a) separar temas o colas por tipo de evento y dar permiso de publicación sólo al servicio dueño de cada uno; (b) firmar los eventos críticos con una clave asimétrica de KMS y verificar la firma en el consumidor; (c) consumidores idempotentes y una cola de mensajes fallidos (DLQ). Si usan Kafka (MSK) en lugar de SQS, habiliten autenticación de clientes con mTLS o IAM y ACLs por tópico.
>
> **Observación 2 — La ventana de 300 s no basta contra la repetición de webhooks.** La ventana impide reenviar notificaciones viejas, pero dentro de esos 5 minutos el mismo evento firmado puede llegar varias veces (Stripe incluso reintenta de forma legítima). Registren el `event.id` ya procesado e ignoren los repetidos. Además, antes de despachar, consulten el `PaymentIntent` en la API de Stripe y comparen monto y moneda con el pedido: una firma válida sólo prueba que el mensaje viene de Stripe, no que corresponde al pedido correcto.
>
> **Observación 3 — Dónde guarda el navegador los JWT.** El diseño no indica cómo almacena la SPA los tokens. Si se guardan en `localStorage`, cualquier XSS puede leerlos y reutilizarlos desde otra máquina durante toda su vigencia. Recomiendo guardar la sesión en una cookie `HttpOnly; Secure; SameSite=Strict`, o bien mantener el token de acceso sólo en memoria con vida corta y el de refresco en una cookie `HttpOnly` con rotación en cada uso.

Análisis crítico del equipo

Las tres observaciones son técnicamente correctas y aplican a nuestro diseño. Las evaluamos una por una en la sección "IA como Evaluador": la observación 2 se aceptó tal cual y las observaciones 1 y 3 se modificaron para ajustarlas a la arquitectura.

Decisión final

🟨 Observación 1: Modificada · ✅ Observación 2: Aceptada · 🟨 Observación 3: Modificada

Qué cambió en el proyecto: secciones 5, 9, 11 y 16 del documento técnico, diagrama de arquitectura (bus SNS → SQS, `PedidoPagado` publicado y firmado por Pagos) y riesgo R6 de la matriz.